



**Ciências
ULisboa**

Project Report

Course: Cloud Computing

Academic Year: 2025/2026

Prepared by:

Tiago Pina (66101)

Leonor Silva (59811)

Tiago Pereira (55854)

Joana Carrasqueira (64414)

Lisbon, June 2026

Table of Contents

1. Introduction.....	1
1.1. Motivation.....	1
1.2. Dataset Characterization.....	1
1.3. Business Capabilities.....	2
1.4. Use Cases.....	2
2. API Specification.....	3
3. Functional and Non-Functional Requirements.....	6
3.1. Functional Requirements.....	6
3.2. Non-Functional Requirements.....	8
4. Architecture.....	9
4.1. Application Architecture.....	9
4.2. Technical Architecture.....	10
5. Implementation.....	12
5.1. Technology Stack.....	12
5.2. Authentication Implementation.....	12
5.3. Review System and Async Processing.....	12
5.4. Recommendations.....	13
5.5. Inter-Service Communication.....	13
6. Deployment Scripts and Tuning.....	13
6.1. Deployment Script.....	13
6.2. Kubernetes Manifests.....	14
7. CI/CD Pipeline.....	15
7.1. Users-service and Movies-service (Leonor Silva).....	15
7.2. Reviews-service and Recommendations-service (Joana Carrasqueira).....	17
7.3. Badges-service and Watchlists-service (Tiago Pina).....	21
7.4. Terraform and Ansible Deployment Workflows (Tiago Pereira).....	23
8. Test and Evaluation.....	25
8.1. GET /users and POST /users (Leonor Silva).....	25
8.1.1. Stress Test (GET /users).....	25
8.1.2. Smoke Test (POST /users).....	27
8.2. GET /anime and PUT /users/{user_id} (Joana Carrasqueira).....	29
8.2.1. Stress Test (GET /anime).....	29
8.2.2. Smoke Test (PUT /users/{user_id}).....	30
8.3. GET /anime/search and DELETE /rating/{id} (Tiago Pina).....	34
8.3.1. Stress Test (GET /anime/search).....	34
8.3.2. Smoke Test (DELETE /rating/{id}).....	36
8.4. GET /rating and POST /rating (Tiago Pereira).....	37
8.4.1. Stress Test (GET /rating).....	37
8.4.2. Smoke Test (POST /rating).....	40
9. Corrections Implemented After Feedback.....	41
10. Cost Analysis.....	42
10.1. Monthly Cost.....	42
10.2. FinOps Measures.....	42
11. Conclusions.....	44

1. Introduction

1.1. Motivation

The project consists of a cloud-native movie platform that provides a unified movie catalog, a public review system, personal watchlists, and a personalized recommendation system. The objective was to build and deploy a scalable analytics and recommendation platform on publicly available movie datasets, leveraging GCP/GKE infrastructure and a microservices architecture. The platform also incorporates AI-powered features, namely asynchronous sentiment analysis of user reviews and natural language generation of personalised movie recommendations, both powered by Google Vertex AI (Gemini 2.5 Flash).

1.2. Dataset Characterization

The platform's data was sourced from two datasets: the *MovieLens 25M* dataset, published by GroupLens Research, as the primary source, and the *IMDb Non-Commercial* Datasets as an auxiliary source for movie metadata enrichment.

MovieLens 25M contains 25 million ratings applied to 62,424 movies by 162,541 users, distributed across three CSV files: `ratings.csv`, `movies.csv`, and `tags.csv`. It provided the core rating and catalog data for the platform.

IMDb Non-Commercial datasets provided supplementary metadata not available in MovieLens, including movie descriptions, runtime, parental rating, and the IMDb URL used to construct external links. Four files were used: `title.basics.tsv.gz` (around 1 GB), `title.ratings.tsv.gz` (27.3 MB), `name.basics.tsv.gz` (892 MB), and `title.principals.tsv.gz` (4.08 GB). The dataset is updated daily and the version used was released March 18, 2024.

Dataset	Source	Size	Format	Records
MovieLens 25M	GroupLens	1.07 GB	CSV	25M ratings, 62,424 movies, 162,541 users
IMDb Non-Commercial	IMDb	~6 GB (compressed)	TSV	Title metadata, ratings, people, principals

Table 1 - Dataset used

Due to infrastructure and performance constraints, only a subset of the full dataset was loaded into the production databases. Specifically, 858,545 ratings were imported (approximately 3.4% of the total), all 62,424 movies with their titles, release years, and genres were loaded, and all 162,541 user identifiers were preserved to maintain referential integrity with the original rating data. IMDb data was used to enrich movie records with descriptions, runtime values, parental ratings, and IMDb URLs, matched to MovieLens entries.

Since neither dataset includes user profile attributes or platform-specific entities such as watchlists, badges, and subscriptions, these were synthetically generated. User attributes (username, email, password, age, gender) were produced using the Python Faker library. Movie cast entries were populated from the IMDb dataset using `title.principals.tsv.gz` for the movie-person relationships and `name.basics.tsv.gz` for the corresponding people metadata. Platform entities such as badges, user badge assignments, watchlist entries, and

subscriptions were populated with randomised mock data to enable end-to-end testing of all microservices.

1.3. Business Capabilities

BC1 — User Identity & Access Management

Centralised authentication and authorisation layer powered by Keycloak, supporting username/password login with JWT tokens and role-based access control (user/admin). Provides a secure, unified identity foundation for all platform services. Positioned as a reusable authentication backbone extensible to third-party integrations via OAuth2.

BC2 — Movie Catalog

Comprehensive movie catalog with full CRUD capabilities restricted to administrators, supporting soft-delete to preserve historical data integrity. Enables browsing, filtering by genre and year, and full-text search with pagination and sorting. The catalog serves as the central content layer consumed by all other platform services.

BC3 — Subscription System

A subscription plan with two tiers (free and premium) providing direct recurring revenue. Premium users gain access to advanced analytics for viewing habits and beta access to new platform features. Revenue model: monthly or yearly premium subscriptions.

BC4 — Review & Rating System

Public review and rating system allowing users to evaluate movies on a 1.0–5.0 scale with optional written reviews. Feeds directly into the recommendation engine and AI analysis pipeline.

BC5 — Smart Intelligent Review System

AI-powered system performing automatic sentiment analysis on user reviews, generating summaries of recurring strengths and weaknesses, and identifying topics via LDA. Increasing user engagement.

BC6 — Personalised Recommendation Engine

Recommendation engine driven by user genre preferences and reference movie history, generating personalised movie suggestions with natural language explanations powered by Vertex AI (Gemini 2.5 Flash). Increases content discovery and session depth.

BC7 — Watchlist Management

Personal watchlist system allowing users to organise movies into named lists for future viewing. Supports multiple concurrent watchlists per user and serves as a behavioural signal for the recommendation engine.

BC8 — Engagement Badges System

Exclusive badges awarded upon completing platform usage milestones (number of ratings submitted, genre exploration, watchlist creation), driving retention and time-on-platform. The module is also positioned as a white-label solution for other content platforms.

1.4. Use Cases

The platform supports 12 use cases spanning user management, content discovery, community contribution, and intelligent personalisation.

UC	Name	Actor	Description
UC1	User Registration	Anonymous User	Register with username, email, and password. System validates uniqueness and password rules, creates an account, and assigns user role in Keycloak.
UC2	User Authentication	Registered User	Authenticate with username and password. System returns a JWT access token (15 min) and refresh token via Keycloak.
UC3	User Profile Management	Authenticated User/ Admin	Update profile attributes (username, gender, age). Admins can list all users or delete any account.
UC4	Browse & Search Movies	Any User	Browse the movie catalog with pagination, sorting, and filters by genre and year. Keyword search across movie titles.
UC5	Movie Management	Admin	Create, update, or soft-delete a movie. Deleted movies are hidden from public endpoints, but associated data is preserved.
UC6	Submit Rating & Review	Authenticated User	Submit a rating (1.0–5.0) with optional review text. If review is provided, triggers asynchronous AI analysis via Pub/Sub.
UC7	Asynchronous Review Analysis	System (review-worker)	Consume Pub/Sub message, invoke Vertex AI to classify sentiment and extract up to 5 topics, and persist results to the database.
UC8	View Review Summary	Any User	View aggregated sentiment distribution and top topics for a movie, read from pre-processed database results.
UC9	Get Personalised Recommendations	Authenticated User	Request ranked movie recommendations with natural language explanations generated by Vertex AI based on user preferences.
UC10	Manage Watchlist	Authenticated User	Create named watchlists and add or remove movies. Multiple concurrent watchlists per user are supported.
UC11	Manage Subscription	Authenticated User	Activate, upgrade, or cancel a subscription plan (free or premium). One active subscription per user.
UC12	Award & View Badges	Authenticated User/ Admin / System	View badges held by a user. Admins create or delete badge definitions. System awards badges upon milestone completion. SSE stream for real-time events.

Table 2 - Use Case Summary

2. API Specification

The platform exposes 7 independent REST APIs, all served over HTTPS via an NGINX Ingress Controller and documented through Swagger UI at /<service>/docs. All write operations and personalised endpoints require a Bearer JWT token obtained from Keycloak.

Service	Method	Endpoint	Auth	Description
User	POST	/users-service/users	Public	Register a new user
	POST	/users-service/login	Public	Authenticate and obtain a JWT token
	GET	/users-service/users	Public	List users with filters
	GET	/users-service/users/{user_id}	Public	Get user profile
	PUT	/users-service/users/{user_id}	User	Update own profile
	DELETE	/users-service/users/{user_id}	User	Delete own account
	PATCH	/users-service/users/{user_id}/admin	Admin	Grant or revoke admin role
Movies	GET	/movies-service/movies	Public	List movies with filters and pagination
	POST	/movies-service/movies	Admin	Create movie
	GET	/movies-service/movies/{movie_id}	Public	Get movie detail
	PUT	/movies-service/movies/{movie_id}	Admin	Update movie
	DELETE	/movies-service/movies/{movie_id}	Admin	Soft-delete movie
	GET	/movies-service/genres	Public	List all genres
	GET	/movies-service/movies/{movie_id}/cast	Public	Get the movie cast
	POST	/movies-service/movies/{movie_id}/cast	Admin	Add a cast member
	DELETE	/movies-service/movies/{movie_id}/cast/{cast_id}	Admin	Remove a cast member
Reviews	POST	/reviews-service/ratings	User	Submit rating and optional review
	GET	/reviews-service/ratings	Public	List ratings with filters
	GET	/reviews-service/ratings/{rating_id}	Public	Get rating
	PUT	/reviews-service/ratings/{rating_id}	User	Update own rating
	DELETE	/reviews-service/ratings/{rating_id}	User	Delete own rating
	POST	/reviews-service/movies/{movie_id}/ratings	User	Submit a rating for a specific movie
	GET	/reviews-service/movies/{movie_id}	Public	Get all ratings for a

		/ratings		movie
	GET	/reviews-service/users/{user_id}/ratings	Public	Get all ratings by a user
	GET	/reviews-service/movies/{movie_id}/review-summary	Public	Get aggregated sentiment and topics
Recommendations	POST	/recommendations-service/users/{user_id}/preferences	User	Add genre preference
	GET	/recommendations-service/users/{user_id}/preferences	User	Get genre preferences
	DELETE	/recommendations-service/users/{user_id}/preferences/{genre_id}	User	Remove genre preference
	POST	/recommendations-service/users/{user_id}/reference-movies	User	Add reference movie
	GET	/recommendations-service/users/{user_id}/reference-movies	User	Get reference movies
	DELETE	/recommendations-service/users/{user_id}/reference-movies/{movie_id}	User	Remove reference movie
	GET	/recommendations-service/recommendations/{user_id}	User	Get personalised recommendations
	GET	/recommendations-service/recommendations/{user_id}/explained	User	Get recommendations with AI explanations
Subscription	POST	/subscriptions-service/users/{user_id}/subscription	User	Create subscription
	GET	/subscriptions-service/users/{user_id}/subscription	User	Get subscription
	PUT	/subscriptions-service/users/{user_id}/subscription	User	Update subscription
	DELETE	/subscriptions-service/users/{user_id}/subscription	User	Cancel subscription
Badges	GET	/badges-service/badges	Public	List all badge definitions
	POST	/badges-service/badges	Admin	Create badge definition
	GET	/badges-service/badges/{badge_id}	Public	Get badge
	PUT	/badges-service/badges/{badge_id}	Admin	Update badge
	DELETE	/badges-service/badges/{badge_id}	Admin	Delete badge
	GET	/badges-service/users/{user_id}/badges	Public	List badges awarded to the user

	POST	/badges-service/users/{user_id}/badges	Admin	Award badge to the user
	GET	/badges-service/users/{user_id}/badges/stream	User	SSE stream of badge award events
Watchlists	GET	/watchlists-service/watchlists	Public	List all watchlists
	POST	/watchlists-service/watchlists	User	Create watchlist
	GET	/watchlists-service/watchlists/{watchlist_id}	Public	Get a watchlist with movies
	PUT	/watchlists-service/watchlists/{watchlist_id}	User	Update watchlist title
	DELETE	/watchlists-service/watchlists/{watchlist_id}	User	Delete watchlist
	POST	/watchlists-service/watchlists/{watchlist_id}/movies	User	Add movie to watchlist
	DELETE	/watchlists-service/watchlists/{watchlist_id}/movies/{movie_id}	User	Remove movie from watchlist
	GET	/watchlists-service/users/{user_id}/watchlist	User	Get all watchlists for a user

Table 3 - API Endpoints

3. Functional and Non-Functional Requirements

3.1. Functional Requirements

ID	Requirement	Service
FR1	The system shall allow new users to register with a username, email, and password, enforcing the uniqueness of both fields and password security rules (minimum 15 characters, at least one uppercase letter, one digit, and one special character).	users
FR2	The system shall authenticate users via Keycloak, returning a JWT access token (15 min lifetime) and a refresh token upon successful login.	users
FR3	The system shall allow authenticated users to update their own profile (username, gender, age) and delete their own account.	users
FR4	The system shall support role-based access control with two roles: user and admin, managed through Keycloak realm roles.	users
FR5	The system shall provide a detailed view of a movie by ID, including title, release year, genres, average rating, cast, runtime, parental rating, poster URL, and IMDb URL. Non-existent or soft-deleted movies shall return a 404 error.	movies
FR6	The system shall support keyword search across movie titles, with optional filters by genre and release year, and return paginated, sortable results.	movies

FR7	The system shall provide a paginated list of non-deleted movies, sorted by title, release year, and rating, and filtered by genre and year.	movies
FR8	The system shall allow administrators to create, update, and soft-delete movies. Soft-deleted movies shall be hidden from all public endpoints while preserving associated ratings and watchlist entries for historical integrity.	movies
FR9	The system shall allow authenticated users to submit, update, and delete ratings (1.0–5.0) for movies, with an optional written review and tag. User and movie existence shall be validated via gRPC before persisting.	review
FR10	The system shall provide filterable and paginated listings of ratings by user, movie, rating range, and tag.	review
FR11	The system shall recalculate and update the average rating of a movie in the movies database whenever a rating is submitted, updated, or deleted.	review/movies
FR12	Upon submission of a review with text, the system shall asynchronously analyse it using Vertex AI to classify sentiment (positive, negative, or neutral) and extract up to 5 topics, processing submissions via a Pub/Sub pipeline without blocking the submission response.	review / review-worker
FR13	The system shall provide an aggregated review summary per movie, including sentiment distribution and most frequent topics, read from pre-processed database results.	review
FR14	The system shall allow administrators to create, update, and delete badge definitions, each with a title, milestone threshold, and optional description.	badges
FR15	The system shall allow administrators to award badges to users upon reaching defined milestones. Duplicate awards for the same badge and user shall be rejected.	badges
FR16	The system shall allow any user to retrieve all badges awarded to a specific user. Badge award events shall be streamable in real time via SSE.	badges
FR17	The system shall allow authenticated users to create named watchlists. User existence shall be validated via gRPC before creation.	watchlists
FR18	The system shall allow authenticated users to update the title of an existing watchlist and to add or remove movies from it.	watchlists
FR19	The system shall allow authenticated users to delete a watchlist and all its associated movie entries.	watchlists
FR20	The system shall allow authenticated users to retrieve all watchlists belonging to a specific user, including the movies contained in each list.	watchlists
FR21	The system shall allow authenticated users to activate a subscription plan (free or premium), with one active subscription per user. User existence shall be validated via gRPC before creation.	subscriptions
FR22	The system shall allow authenticated users to upgrade, downgrade, or update the status of their subscription (active, pending, cancelled, expired), including start and end dates for premium plans.	subscriptions

FR23	The system shall allow authenticated users to create, retrieve, update, and delete their subscription record.	subscriptions
FR24	The system shall allow authenticated users to manage genre preferences and reference movies, used as input to the recommendation engine	recommendations
FR25	The system shall generate personalised movie recommendations with optional natural language explanations powered by Vertex AI (Gemini 2.5 Flash)	recommendations

Table 4 - Functional Requirements Summary

3.2. Non-Functional Requirements

ID	Requirement	Implementation
NFR1	Each microservice shall be independently deployable and maintain its own isolated PostgreSQL database, following the database-per-service pattern.	7 separate PostgreSQL, 15 instances, one per service
NFR2	All synchronous inter-service communication shall use gRPC, and asynchronous event-driven processing shall use Google Cloud Pub/Sub.	gRPC (ports 50053–50059), Pub/Sub topic review-created
NFR3	All API endpoints shall be served over HTTPS with TLS certificates issued automatically by Let's Encrypt.	cert-manager, NGINX Ingress Controller, nip.io hostname
NFR4	Authentication tokens shall be validated using RS256 asymmetric signatures, with public keys fetched from Keycloak's JWKS endpoint.	Keycloak 24, JWT RS256, python-jose
NFR5	All sensitive configuration values shall be stored externally and injected into pods at runtime, with no credentials hardcoded in manifests or images.	GCP Secret Manager, External Secrets Operator, Workload Identity
NFR6	Network traffic between pods shall follow a default-deny policy with explicit allow rules per service and per port.	14 Kubernetes NetworkPolicy rules
NFR7	The ingress shall enforce rate limiting to protect services from abusive traffic patterns.	NGINX rate limiting: 10 req/s, burst multiplier 5x, max 10 connections
NFR8	Each microservice shall expose metrics for scraping by Prometheus, with dashboards available in Grafana.	prometheus-fastapi-instrumentator, Prometheus + Grafana stack
NFR9	Each microservice shall expose a health endpoint used by Kubernetes liveness and readiness probes to ensure traffic is only routed to healthy pods.	FastAPI /health, Kubernetes livenessProbe, and readinessProbe
NFR10	The review analysis pipeline shall be non-blocking — the review submission endpoint shall return HTTP 201 immediately, with AI processing occurring asynchronously.	Pub/Sub async pipeline, dedicated review-worker deployment
NFR11	Pod resource requests and limits shall be defined for all deployments to ensure predictable scheduling and	requests and limits defined in all Kubernetes

	prevent resource contention.	Deployment YAMLs
NFR12	The cluster shall be deployed in a single GCP region to minimise latency between services and reduce data transfer costs.	GKE cluster in europe-west1-b
NFR13	The platform shall be deployable via a single automated script requiring no manual kubectl intervention beyond initial GCP setup.	deploy.sh — fully automated 17-step deployment

Table 5 - Non-Functional Requirements Summary

4. Architecture

4.1. Application Architecture

The platform follows a microservices architecture pattern, decomposing the system into 8 independently deployable services, each with a single responsibility and its own isolated PostgreSQL database. There is no shared database between services — all cross-service data access is performed via explicit API calls, either REST (for client-facing operations) or gRPC (for internal service-to-service communication).

The application layer is organised as follows:

Service	Responsibility	REST Port	gRPC Port	Database
users-service	User registration, authentication, and profile management	8001	50053	postgres_users
movies-service	Movie catalog, genres, and cast management	8002	50054	postgres_movies
review-service	Ratings, reviews, fraud detection, and sentiment summaries	8003	50055	postgres-ratings
review-worker	Asynchronous review analysis via Pub/Sub and Vertex AI	—	—	postgres-ratings
recommendations-service	Genre preferences, reference movies, and personalised recommendations	8004	50056	postgres-recommendations
subscriptions-service	Subscription plan management	8005	50057	postgres-subscriptions
badges-service	Badge definitions, awards, and SSE event streaming	8006	50058	postgres-badges
watchlists-service	User watchlist management	8007	50059	postgres-watchlists

Table 6 - Microservices

Inter-service communication uses two patterns. gRPC is used for synchronous calls. User and movie existence validation before persisting records, and average rating recalculation pushed from review-service to movies-service after every rating change. GCP Pub/Sub is used exclusively for the asynchronous review analysis pipeline, where review-service publishes a message upon review submission and review-worker independently processes it via Vertex AI without blocking the original request.

Service (caller)	Calls (via gRPC)
review-service	users-service, movies-service
recommendations-service	users-service, movies-service, review-service
badges-service	users-service
subscriptions-service	users-service
watchlists-service	users-service

Table 7 - gRPC Dependencies

4.2. Technical Architecture

The platform is deployed on Google Kubernetes Engine (GKE) in the europe-west1-b zone.

Cluster configuration:

- 3 nodes, machine type e2-standard-2 (2 vCPU, 8 GB RAM each)
- Disk: 30 GB per node
- Workload Identity enabled for GCP service account access
- Network Policy enabled

Component	Technology	Purpose
Container Orchestration	GKE (Kubernetes)	Pod scheduling, scaling, and self-healing
Ingress	NGINX Ingress Controller	HTTP/HTTPS routing, path-based dispatch, rate limiting
TLS	cert-manager + Let's Encrypt	Automatic certificate provisioning
Authentication	Keycloak 24	JWT issuance and validation (RS256)
Secret Management	External Secrets Operator + GCP Secret Manager	Inject secrets into pods without hardcoding
Metrics	Prometheus + Grafana	Service-level metrics and dashboards
Async Messaging	GCP Pub/Sub	Decoupled review analysis pipeline
AI/ML	Vertex AI (Gemini 2.5 Flash)	Sentiment analysis, topic extraction, and recommendation explanations
Image registry	Docker Hub	Container image storage and versioning

Table 8 - Infrastructure Components

Each microservice has a dedicated PostgreSQL 15 instance, ensuring data ownership boundaries and independent schema evolution. Storage volumes are sized according to data volume requirements; postgres-movies and postgres-ratings are allocated 8Gi each to accommodate the MovieLens dataset, while the remaining instances are allocated 2Gi.

Database	Storage
postgres-movies	8Gi
postgres-ratings	8Gi
postgres-users	2Gi
postgres-recommendations	2Gi
postgres-subscriptions	2Gi
postgres-badges	2Gi
postgres-watchlists	2Gi

Table 9 - PostgreSQL Storage Allocation

Network Policies enforce strict traffic rules: only the Ingress can reach REST ports, services communicate only via declared gRPC ports, and PostgreSQL instances only accept connections from their corresponding service and dedicated postgres-exporter pod. External HTTPS egress (port 443) is permitted for GCP API calls (Pub/Sub, Vertex AI, Secret Manager).

All microservices run with a single replica (`replicas: 1`). Horizontal scaling was not configured in the current deployment and is identified as a future improvement.

Service	CPU Request	CPU Limit	Memory Request	Memory Limit
users-service	100m	300m	128Mi	256Mi
movies-service	100m	300m	128Mi	256Mi
review-service	100m	300m	128Mi	256Mi
review-worker	100m	300m	256Mi	512Mi
recommendations-service	200m	500m	512Mi	1Gi
subscriptions-service	100m	300m	128Mi	256Mi
badges-service	100m	300m	128Mi	256Mi
watchlists-service	100m	300m	128Mi	256Mi

Table 10 - Resource Allocation per Pod

The recommendations-service is allocated more CPU and memory due to Vertex AI SDK overhead and in-memory preference vector computations. The review-worker has increased memory allocation relative to other services as it processes Pub/Sub message payloads and invokes the Vertex AI SDK for each review analysed. Keycloak runs in `start-dev` mode with an embedded H2 database, which is appropriate for the current deployment scale but would need to be replaced with a dedicated PostgreSQL backend for production use.

5. Implementation

5.1. Technology Stack

All microservices are implemented in Python using FastAPI as the web framework, with SQLAlchemy as the ORM layer and psycopg2 for PostgreSQL access. gRPC stubs are generated from .proto files using grpcio-tools. Prometheus metrics are automatically instrumented via prometheus-fastapi-instrumentator.

Layer	Technology
Web Framework	FastAPI (Python)
ORM	SQLAlchemy 2.0
Database Driver	psycopg2-binary
gRPC	grpcio 1.78
JWT Validation	python-jose[cryptography] with RS256/JWKS
HTTP Client	httpx
Metrics	prometheus-fastapi-instrumentator
AI Integration	google-genai (Vertex AI)
Async Messaging	google-cloud-pubsub

Table 11 - Technology Stack

5.2. Authentication Implementation

Authentication is implemented directly in each microservice rather than at the gateway level. Each service fetches the Keycloak JWKS endpoint at runtime and validates the RS256 signature of incoming Bearer tokens. Two protection levels are applied:

- `require_auth` — validates token signature and issuer; any authenticated user passes
- `require_admin` — additionally verifies the presence of the `admin` role in `realm_access.roles`

This approach was chosen to allow fine-grained per-endpoint control without a dedicated API gateway service.

5.3. Review System and Async Processing

The review-service exposes REST endpoints for rating CRUD and delegates AI-powered analysis to the review-worker via GCP Pub/Sub. When a rating is submitted or updated with a written review text, an event is published to the `review-created` topic. The review-worker consumes these events, calls Vertex AI (Gemini 2.5 Flash) for sentiment classification (positive, negative, or neutral) and extraction of up to 5 topics, and writes results back to the ratings database. The submission endpoint returns HTTP 201 immediately. The analysis pipeline is fully non-blocking.

Additionally, whenever a rating is created, updated, or deleted, the review-service recalculates the average rating for the affected movie and pushes the updated value to the movies-service via gRPC.

5.4. Recommendations

The recommendations-service stores per-user genre preferences and reference movie selections. Recommendations are generated by querying the Vertex AI Gemini 2.5 Flash API with the user's preference profile and rating history, returning a ranked list of movie suggestions. An explained variant invokes the same model to produce a natural language justification for each recommendation.

5.5. Inter-Service Communication

gRPC is used for synchronous cross-service calls that require immediate responses:

- Validating user existence before creating a subscription, watchlist, or badge award
- Validating movie existence before submitting a rating
- Fetching movie metadata and ratings within the recommendations-service
- Pushing recalculated average ratings from review-service to movies-service

Each service exposes a gRPC server on a dedicated port (50053–50059). gRPC channels are managed as persistent connections initialised at service startup via the FastAPI lifespan event. Notably, the badges-service and watchlists-service follow an internal REST/gRPC separation pattern — the REST layer communicates with its own gRPC server for all data operations, in addition to calling the users-service externally for user validation.

6. Deployment Scripts and Tuning

6.1. Deployment Script

The `deploy.sh` script automates the full application deployment onto an existing GKE cluster. The cluster itself must be created manually as a one-time setup step, as described in `deploy_guide.md`. The script handles the following:

- Connecting `kubectl` to the GKE cluster via `gcloud container clusters get-credentials`
- Installing `cert-manager`, `NGINX Ingress Controller`, and `External Secrets Operator` via `Helm` (all idempotent)
- Configuring the `GCP Secret Manager` store and syncing secrets into `Kubernetes` via `ESO`
- Creating the `gcp-sa-secret` from `sa-key.json` for `Vertex AI` and `Pub/Sub` access
- Applying all `Kubernetes` manifests in dependency order
- Waiting for each component to reach a healthy state before proceeding
- Detecting the `NGINX LoadBalancer` IP and applying the `TLS Ingress` with a `nip.io` hostname

The script exits immediately on any error (`set -e`) and uses `--ignore-not-found` on all delete commands to support both clean and re-deployment scenarios.

Note: the script uses `Linux` line endings (`LF`). On `Windows` environments, `sed -i 's/\r//'` `deploy.sh` must be run before execution.

6.2. Kubernetes Manifests

File	Contents
00-configmap.yaml	ConfigMap with service URLs, ports, and GCP project configuration
01-secret-store.yaml	ESO SecretStore pointing to GCP Secret Manager
01-external-secret.yaml	ExternalSecret syncing DB credentials, Keycloak secret, and admin password
02-postgres-*.yaml	PostgreSQL Deployment + PVC + ClusterIP Service (7 files, one per service)
03-populate-db-*.yaml	Kubernetes Jobs to seed each database from CSV data (7 files)
04-users-service.yaml	Users deployment + ClusterIP Service
05-movies-service.yaml	Movies deployment + ClusterIP Service
06-reviews-service.yaml	Review system deployment + ClusterIP Service
07-review-worker.yaml	Review worker deployment (Pub/Sub consumer)
08-recommendations-service.yaml	Recommendations deployment + ClusterIP Service
09-badges-service.yaml	Badges deployment + ClusterIP Service
10-subscriptions-service.yaml	Subscriptions deployment + ClusterIP Service
11-watchlists-service.yaml	Watchlists deployment + ClusterIP Service
12-keycloak.yaml	Keycloak StatefulSet + ClusterIP Service
13-keycloak-setup-job.yaml	Kubernetes Jobs for Keycloak realm setup and admin user creation
14-network-policies.yaml	All 14 NetworkPolicy rules
15-cluster-issuer.yaml	Let's Encrypt ClusterIssuer for cert-manager
16-monitoring-postgres.yaml	postgres-exporter Deployments (one per database)
17-monitoring-bridge.yaml	ExternalName Services bridging monitoring namespace to default
18-monitoring-scrape.yaml	Prometheus ScrapeConfig and additional scrape configs Secret
19-ingress.yaml	NGINX Ingress with TLS, path-based routing, and rate limiting

Table 12 - Kubernetes Manifest Files

7. CI/CD Pipeline

7.1. Users-service and Movies-service (Leonor Silva)

Two independent GitHub Actions pipelines were implemented, covering the users-service and movies-service (`ci_cd_users.yml` and `ci_cd_movies.yml`). Both pipelines follow an identical 5-stage sequential structure where each stage only executes if the previous one passes. Pipelines trigger on every push to the `lsilva` branch and on every Pull Request targeting that branch. The Docker push stage (Stage 5) is restricted to direct pushes only. Pull Requests trigger stages 1–4 but do not publish images.

Stage	Name	Tool	Description
1	Lint	flake8	Static code analysis with max line length 120, excluding test and gRPC directories
2	Unit Tests	pytest	Model validator tests in complete isolation — no database, no external services
3	Integration Tests	Pytest + SQLite	FastAPI endpoint tests via TestClient with SQLite database and mocked external calls
4	Build	Docker Buildx	Builds the Docker image without pushing; uses GitHub Actions cache for layer reuse
5	Push	Docker Hub	Builds and pushes an image tagged with the latest and <code>{{ github.sha }}</code> to Docker Hub

Table 13 - CI/CD Pipeline Stages

Unit tests validate Pydantic model validators in complete isolation; no database connection, no Keycloak, no network calls.

Service	Test Class	Tests
users-service	TestPasswordValidation	Too short (<15 chars), no uppercase, no digit, no special char, valid password accepted, exactly 15 chars accepted
	TestEmailValidation	Missing @, missing domain, valid email accepted
	TestGenderValidation	All four valid codes (M, F, NB, O) accepted, invalid code rejected, optional field (None) accepted
	TestTermsValidation	termsAccepted=False rejected, termsAccepted=True accepted
	TestUserUpdateValidation	Invalid gender rejected, empty update (all None) accepted
movies-service	TestReleaseYearValidation	Year before 1874 rejected, exactly 1874 accepted, modern year accepted, invalid year in update rejected, None in update accepted
	TestRuntimeValidation	Zero rejected, negative rejected, positive accepted, None accepted

	TestMovieCreateRequiredFields	Missing title rejected, missing release year rejected, minimal valid movie accepted
	TestMovieUpdateValidation	Empty update accepted, partial update accepted

Table 14 - Unit Tests

Integration tests exercise the full FastAPI request/response cycle using a SQLite in-memory database. The DB_URL environment variable is overridden before any app module is imported, causing config.py to use SQLite instead of PostgreSQL. Each test cleans up its data after execution to ensure full isolation between runs.

Service	Test Class	Tests
users-service	TestHealth	GET /health returns 200 and {"status": "ok"}
	TestUserRegistration	Successful registration (201), duplicate username (409), duplicate email (409), weak password (422), terms not accepted (422)
	TestGetUser	Existing user (200), non-existent ID (404)
	TestListUsers	Returns a list, includes the registered user, filters by username, and invalid gender returns 400
	TestLogin	Successful login returns a token, invalid credentials return 401
	TestUpdateUser	Own profile updated (200), different user returns 403, non-existent user returns 404
	TestDeleteUser	Own account deleted (204), different user returns 403, non-existent user returns 404
	TestSetAdmin	Promote to admin (200), revoke admin (200), non-admin caller returns 403
	TestNFR01PasswordStorage	The password field in the DB contains managed-by-keycloak, never the plain-text credential
	TestNFR02AsymmetricTokenSigning	HS256-forged token rejected with 401 — only RS256 tokens accepted
movies-service	TestHealth	GET /health returns 200 and {"status": "ok"}
	TestGenres	Empty DB returns empty list, seeded genre appears in list
	TestListMovies	Empty DB returns empty list, created movie appears, filter by title, filter by

		year, filter by genre
	TestCreateMovie	Successful creation (201), with genre association, without token returns 403, missing title returns 422
	TestGetMovie	Existing movie (200), non-existent ID (404)
	TestDeleteMovie	Soft-deleted movie hidden from the list, without a token returns 403, non-existent ID returns 404
	TestCast	Empty cast list, add cast member (201), remove cast member (204)
	TestUpdateMovie	Title and runtime updated, genre list replaced, non-existent ID returns 404, without token returns 403

Table 15 - Integration Tests

All Keycloak interactions are mocked at the boundary using `pytest-mock` and `unittest.mock.patch`, so application logic is fully exercised without requiring a running Keycloak instance.

Scenario	What is mocked
POST /login	<code>httpx.post</code> returns a fake token response
POST /users (register)	<code>_create_keycloak_user</code> skips the HTTP call to Keycloak
PUT /users/{id} (update)	<code>decode_keycloak_token</code> returns a controlled payload
DELETE /users/{id}	<code>decode_keycloak_token</code> + <code>_delete_keycloak_user</code> both patched
PATCH /users/{id}/admin	<code>decode_keycloak_token</code> + <code>_assign_keycloak_role</code> / <code>_remove_keycloak_role</code> patched
Movies admin routes	<code>app.dependency_overrides[require_admin]</code> always passes

Table 16 - Keycloak Mocking Strategy

GitHub secrets required: LSILVA_DOCKERHUB_USERNAME, LSILVA_DOCKERHUB_TOKEN.

7.2. Reviews-service and Recommendations-service (Joana Carrasqueira)

Two independent GitHub Actions pipelines were implemented, one handles the reviews-service and review-worker, while the other handles the recommendations-service (`ci_cd_review.yaml` and `ci_cd_recommendations.yaml`). Both pipelines follow an identical 5-stage sequential structure where each stage only executes if the previous one passes. Pipelines trigger on every push to the `jcarrasqueira` branch and on every Pull Request targeting that branch.

Stages	Description	Notes
Stage 1 - Lint	Runs flake8 on all service folders, skipping generated gRPC stubs. Catches syntax errors and code quality issues before any tests run.	
Stage 2 - Unit Test	Tests Pydantic model validation and pure business logic in isolation. No database, no Docker, no GCP.	Each test file is named test_unit.py and is stored in each service/worker tests folder
Stage 3 - Integration Tests	Tests real endpoint handlers against SQLite. GCP and gRPC clients are mocked; the tests assert they are called correctly, not that they connect to real infrastructure.	Each test file is named test_integration.py and is stored in each service/worker tests folder
Stage 4 - Build	Builds Docker images for each service using the actual Dockerfile in each directory.	This stage only verifies that the image compiles.
Stage 5 - Push to Docker Hub	Rebuilds and pushes the images to Docker Hub. Images are tagged with both the latest and the full commit SHA for traceability.	Only runs on direct pushes to jcarrasqueira.

Table 17 - Stages of Pipeline

For Stage 2, the following tests were implemented:

Service	Test	Description
reviews-service	test_valid_rating_accepted	Valid rating value is accepted
	test_rating_above_max_raises	Rating above 5.0 raises ValidationError
	test_rating_below_min_raises	Rating below 0.0 raises ValidationError
	test_rating_exactly_zero_accepted	Boundary value 0.0 is accepted
	test_rating_exactly_five_accepted	Boundary value 5.0 is accepted
	test_missing_user_id_raises	Missing user_id raises ValidationError
	test_missing_movie_id_raises	Missing movie_id raises ValidationError
	test_missing_rating_raises	Missing rating raises ValidationError
	test_review_defaults_none	Optional review field defaults to None
	test_tag_defaults_none	Optional tag field defaults to None
	test_empty_update_accepted	Empty update body is accepted, all fields optional
	test_partial_update_accepted	Partial update with some fields is

		accepted
	test_rating_above_max_raises	Update rating above 5.0 raises ValidationError
	test_rating_below_min_raises	Update rating below 0.0 raises ValidationError
	test_valid_rating_accepted	Valid movie rating value is accepted
	test_rating_above_max_raises	Movie rating above 5.0 raises ValidationError
	test_rating_below_one_raises	Movie rating below 1.0 raises ValidationError, stricter minimum
	test_missing_user_id_raises	Missing user_id in movie rating raises ValidationError
review-worker	test_process_review_returns_sentiment_and_topics	Vertex AI response is correctly parsed into sentiment and topics
	test_process_review_normalises_topics_to_lowercase	Topics are lowercased and stripped regardless of Vertex AI output
	test_callback_nacks_on_malformed_message	Invalid JSON in message payload triggers nack, not ack
	test_callback_nacks_when_process_review_raises	Vertex AI failure triggers nack, not ack
	test_save_review_handles_db_error_gracefully	DB error triggers rollback and does not raise an exception
recommendations-service	test_valid_like_accepted	Valid like preference is accepted
	test_valid_dislike_accepted	Valid dislike preference is accepted
	test_default_preference_type_is_like	preference_type defaults to like when omitted
	test_missing_genre_id_raises	Missing genre_id raises ValidationError
	test_genre_id_must_be_integer	Non-integer genre_id raises ValidationError
	test_valid_preference_accepted	Valid preference with all fields is accepted
	test_missing_user_id_raises	Missing user_id raises ValidationError
	test_missing_genre_id_raises	Missing genre_id in UserPreference raises ValidationError
	test_default_preference_type	preference_type defaults to like when omitted in UserPreference

Table 18 - Unit Tests

For the Stage 3 the following integration tests were established:

Service	Test	Description
reviews-service	test_health	GET /health returns status ok
	test_creates_rating_successfully	POST /ratings creates a rating and returns 201
	test_creates_rating_and_publishes_when_review_present	Pub/Sub is called once when review text is provided
	test_does_not_publish_when_no_review	Pub/Sub is not called when review text is absent
	test_returns_404_for_nonexistent_user	Unknown user_id returns 404
	test_returns_404_for_nonexistent_movie	Unknown movie_id returns 404
	test_get_existing_rating	GET /ratings/{id} returns the correct rating
	test_get_nonexistent_rating_returns_404	Unknown rating_id returns 404
	test_delete_existing_rating	DELETE /ratings/{id} removes rating and subsequent GET returns 404
	test_delete_nonexistent_rating_returns_404	Unknown rating_id returns 404
	test_update_existing_rating	PUT /ratings/{id} updates the rating value
review-worker	test_save_review_persists_sentiment_to_db	save_review writes sentiment label to review_sentiment table
	test_callback_acks_message_and_persists_to_db	Valid message is processed, acknowledged, and sentiment persisted to DB
recommendations-service	test_health	GET /health returns status ok
	test_creates_preference_successfully	POST /users/{id}/preferences creates preference and returns 200
	test_returns_404_for_nonexistent_user	Unknown user_id returns 404 on preference creation
	test_returns_404_for_nonexistent_genre	Unknown genre_id returns 404
	test_duplicate_preference_returns_400	Duplicate preference returns 400
	test_get_preferences_after_creating	GET /users/{id}/preferences returns created preferences
	test_returns_404_when_no_preference	No preferences returns 404

	s	
	test_returns_404_for_nonexistent_user	Unknown user_id returns 404 on preference retrieval
	test_delete_existing_preference	DELETE /users/{id}/preferences/{genre_id} removes preference
	test_delete_nonexistent_returns_404	Non-existent preference returns 404
	test_add_reference_movie_successfully	POST /users/{id}/reference-movies adds reference movie
	test_duplicate_reference_movie_returns_400	Duplicate reference movie returns 400
	test_returns_404_for_nonexistent_user	Unknown user_id returns 404 on reference movie creation
	test_get_reference_movies_after_adding	GET /users/{id}/reference-movies returns added movies
	test_returns_404_when_none	No reference movies returns 404
	test_calls_vertex_ai_and_returns_explanation	Vertex AI is called once and its response is included in the result
	test_returns_404_for_nonexistent_user	Unknown user_id returns 404 on explained recommendations

Table 19 - Integration Tests

GitHub secrets required:

JCARRASQUEIRA_DOCKERHUB_USERNAME, JCARRASQUEIRA_DOCKERHUB_TOKEN.

7.3. Badges-service and Watchlists-service (Tiago Pina)

The badges-service and watchlists-service each have a dedicated GitHub Actions pipeline (badges-ci-cd.yml and watchlists-ci-cd.yml). Both are structured as two sequential jobs: a CI job (Lint & Test) and a CD job (Build & Push Docker Image), where the Docker job is gated on the CI job passing successfully.

Both pipelines are triggered by pushes to the tpina branch and by Pull Requests targeting it. Image publishing is restricted to direct pushes — Pull Requests execute only the CI job without pushing any image.

Stage	Name	Tool	Description
1	Lint	flake8	Hard fail on syntax errors and undefined names; warnings-only pass for complexity and line length (max 127)
2	Tests	pytest	API endpoint tests via TestClient with both gRPC clients fully mocked using AsyncMock; no real database or network calls
4	Build	Docker Buildx	Builds the Docker image without pushing; uses GitHub Actions cache for layer reuse
5	Push	Docker	Pushes an image tagged with the latest and

		Hub	\${{ github.sha }} to Docker Hub
--	--	-----	----------------------------------

Table 20 - CI/CD Pipeline Stages

Before any tests run, protobuf stubs are generated from .proto files using `grpc_tools.protoc`. Both services compile stubs for their own service proto and for `users/users.proto`, since both depend on the `users-service` gRPC interface to validate user existence. Without this step, module imports would fail before a single test executes.

Test Strategy

Both test suites patch the FastAPI app's two gRPC client globals (`grpc_client` and `user_client`) with `AsyncMock` instances via `unittest.mock.patch`, giving each test full control over returned values without spinning up any gRPC server, PostgreSQL instance, or external service. HTTP calls are driven by Starlette's `TestClient`, which wraps the app directly.

Each test class targets one route group. The `badges-service` suite covers badge definition CRUD, user badge assignment, and the health endpoint. The `watchlists-service` suite covers watchlist CRUD, per-watchlist movie management, user watchlist retrieval, and the health endpoint.

badges-service tests:

Test Class	Tests
<code>TestListBadges</code>	Returns all badge definitions (200), verifies gRPC <code>list_badges</code> was called
<code>TestGetBadge</code>	Returns badge by ID (200), invalid ID (<code>ge=1</code> constraint) returns 422
<code>TestCreateBadge</code>	Successful creation (201), missing title returns 422, missing milestone returns 422, verifies gRPC called with correct args
<code>TestUpdateBadge</code>	Successful update (200), invalid ID returns 422
<code>TestDeleteBadge</code>	Successful deletion (204), invalid ID returns 422
<code>TestGetUserBadges</code>	Returns badges for a given user (200), user not found returns 404
<code>TestAwardBadge</code>	Awards badge (201), user not found returns 404, missing <code>badge_id</code> returns 422, invalid <code>badge_id</code> (<code>ge=1</code>) returns 422
<code>TestHealthCheck</code>	GET <code>/health</code> returns 200 with <code>{"status": "ok", "service": "badges"}</code>

Table 21 - Badges service tests

watchlists-service tests:

Test Class	Tests
<code>TestListWatchlists</code>	Returns all watchlists (200), verifies gRPC <code>list_watchlists</code> was called
<code>TestGetWatchlist</code>	Returns watchlist with embedded movies (200), invalid ID returns 422
<code>TestCreateWatchlist</code>	Successful creation (201), missing title returns 422, missing <code>user_id</code> returns 422, user not found returns 404, verifies gRPC called with correct args
<code>TestUpdateWatchlist</code>	Successful title update (200), missing title returns 422, invalid ID

	returns 422
TestDeleteWatchlist	Successful deletion (204), invalid ID returns 422
TestAddMovie	Adds movie to watchlist (201), missing movie_id returns 422, invalid movie_id or watchlist_id (ge=1) returns 422, verifies gRPC called with correct args
TestRemoveMovie	Removes movie (204), invalid watchlist_id or movie_id returns 422
TestGetUserWatchlists	Returns all watchlists for a given user (200), user not found returns 404

Table 22 - Watchlists service tests

User existence validation is exercised by overriding the `validate_user` mock to return `False` within the relevant test, producing the expected 404 without any real service involvement. Input validation coverage — missing required fields and out-of-range IDs enforced by `ge=1` constraints — is verified by sending intentionally malformed payloads and asserting 422 responses from FastAPI's Pydantic validation layer.

GitHub secrets required: `TPINA_DOCKERHUB_USERNAME`, `TPINA_DOCKERHUB_TOKEN`.

7.4. Terraform and Ansible Deployment Workflows (Tiago Pereira)

Two complementary GitHub Actions workflows were implemented for the infrastructure and deployment layer of the project: one dedicated to Terraform and another dedicated to Ansible. Unlike the application-level CI/CD pipelines developed for the microservices, which focus on code validation, Docker image build, and image publication, these workflows were designed to automate the provisioning of the cloud infrastructure and the configuration of the Kubernetes environment. Together, they replaced the manual setup steps previously required to create the GKE cluster and deploy the Kubernetes resources.

The first workflow is responsible for the infrastructure provisioning phase. Its purpose is to initialise Terraform, validate the configuration files, generate an execution plan, and apply the infrastructure changes required to create or update the GKE environment. In practice, this workflow provisions the cluster and its base configuration directly from GitHub Actions, ensuring that infrastructure changes are reproducible and version-controlled. The second workflow is responsible for the configuration and deployment phase, using Ansible to authenticate to the cluster, install the required Kubernetes dependencies, and apply the manifests and Helm-based components needed for the full platform deployment.

Because both workflows execute remotely in GitHub Actions runners, it was necessary to grant the repository the appropriate Google Cloud permissions. This setup allowed GitHub Actions to authenticate against the target GCP project, manage GKE resources through Terraform, retrieve Kubernetes credentials, and apply the Kubernetes configuration through Ansible without requiring local manual execution. In this sense, the complete deployment workflow was split into two coordinated layers: Terraform handled infrastructure-as-code for cluster creation, while Ansible handled configuration management and application rollout inside the cluster.

Stage	Workflow	Tool	Description
1	Terraform	GCP authentication	Authenticates GitHub Actions to Google Cloud using credentials stored as repository secrets, allowing the workflow to manage cloud resources.
2	Terraform	<code>terraform init</code>	Initializes the Terraform working directory and downloads the required providers and modules.
3	Terraform	<code>terraform validate</code>	Validates the Terraform configuration files before any infrastructure change is applied.
4	Terraform	<code>terraform plan</code>	Generates an execution plan showing the infrastructure changes to be made to the GKE environment.
5	Terraform	<code>terraform apply</code>	Applies the infrastructure changes and creates or updates the GKE cluster and related resources.
6	Terraform / Ansible	<code>gcloud container clusters get-credentials</code>	Retrieves cluster credentials after provisioning so that the deployment workflow can access Kubernetes.
7	Ansible	Dependency setup	Installs the required Ansible collections and Python dependencies needed to manage Kubernetes resources from the workflow runner.
8	Ansible	<code>ansible-playbook</code>	Executes the deployment playbook, which configures the Kubernetes environment and applies the project resources in the correct order.
9	Ansible	Helm + Kubernetes modules	Installs supporting components such as External Secrets Operator, NGINX Ingress, cert-manager, and monitoring, and then deploys the application manifests.
10	Ansible	Verification commands	Confirms that the cluster state is healthy by checking pod rollout, jobs, services, secrets, and ingress configuration.

Table 23 - Terraform and Ansible deployment workflow stages

This separation between Terraform and Ansible introduced a clear distinction between infrastructure provisioning and application deployment. Terraform remained responsible only for the declarative creation of cloud infrastructure, while Ansible was used for imperative orchestration of the Kubernetes setup and deployment logic. This design made the deployment process easier to reason about, easier to debug, and more aligned with the intended role of each technology in the project architecture.

A key aspect of this solution was the integration between GitHub Actions and GCP IAM. In order for the automation to work end-to-end, the GitHub repository had to be configured with secrets and permissions enabling the workflows to operate on the target GCP project. This included permissions to provision GKE resources, retrieve cluster credentials, and perform Kubernetes deployment tasks. Without this configuration, the workflows would have been limited to static validation only and would not have been able to perform the full cluster setup and deployment directly from GitHub Actions.

Item	Type	Purpose
GCP service account credentials	GitHub secret	Used by GitHub Actions to authenticate to the target GCP project and execute Terraform and Ansible steps without manual login.
GCP project identifier	Workflow variable / secret	Identifies the target project where the GKE cluster and supporting resources are created and configured.
GKE cluster name	Workflow variable	Used by both workflows to reference the target Kubernetes cluster during credential retrieval and deployment.
GCP zone / region	Workflow variable	Ensures that Terraform and GKE credential commands target the correct cluster location.
Terraform permissions	IAM roles on the GCP service account	Required so the Terraform workflow can create, update, and manage GKE and related infrastructure resources.
Kubernetes access permissions	IAM + cluster access	Required so the Ansible workflow can retrieve cluster credentials and apply Kubernetes resources to the provisioned cluster.
Helm deployment permissions	Kubernetes RBAC / cluster-admin style access	Required for Ansible to install platform components such as NGINX Ingress, cert-manager, External Secrets Operator, and monitoring.
Repository secrets for deployment	GitHub Actions secrets	Centralized storage for sensitive values required by the workflows, avoiding hardcoded credentials in the repository.

Table 24 - GitHub Actions secrets and GCP permissions required for Terraform and Ansible workflows

8. Test and Evaluation

As part of Phase 9 (Cloudy Day, June 3, 2026), each team member conducted testing against Group 12's deployed API, accessible at <http://34.62.61.34> during the session window (16:00–19:00). Each tester focused on different endpoints. Two types of tests were performed: a stress test using Locust to evaluate throughput and response time under concurrent load, and a smoke test using Postman to validate endpoint behaviour functionally.

8.1. GET /users and POST /users (Leonor Silva)

8.1.1. Stress Test (GET /users)

The stress test evaluated throughput capacity and response time distribution under progressive concurrent load. Users were spawned in 5 phases over 300

seconds using a LoadTestShape, with wait time set to 0 to maximise throughput. The test was run twice to observe performance evolution as other groups added users to the shared database during the session.

Phase	Users	Spawn Rate
1	10	2/s
2	50	5/s
3	100	10/s
4	250	25/s
5	500	50/s

Table 25 - Load Test Phases

Phase	Users	Avg RPS	Max RPS	p50 (ms)	p95 (ms)	p99 (ms)	p99.9 (ms)	Max RT (ms)	Failures
1	10	20.3	23.7	426	612	1,107	1,851	2,700	0%
2	50	70.0	77.0	538	842	1,155	2,306	4,200	0%
3	100	54.0	71.9	642	1,783	9,055	12,535	41,000	0%
4	250	58.4	74.9	767	3,535	19,135	35,212	41,000	0%
5	500	57.8	75.8	1,214	10,433	21,367	36,837	108,000	0%

Table 26 - Stress Test Run 1

Phase	Users	Avg RPS	Max RPS	p50 (ms)	p95 (ms)	p99 (ms)	p99.9 (ms)	Max RT (ms)	Failures
1	10	19.6	22.2	445	601	822	1,321	1,400	0%
2	50	51.9	63.5	596	967	1,504	3,951	27,000	0%
3	100	53.3	65.0	784	1,727	12,367	39,442	57,000	0%
4	250	52.5	65.9	1,263	4,210	20,577	42,154	57,000	0%
5	500	57.6	75.5	1,586	9,118	25,000	44,020	57,000	0%

Table 27 - Stress Test Run 2

Metric	Run 1	Run 2	Change
Total Requests	13,978	10,548	-25%

Peak avg RPS (Phase 2)	70.0	51.9	-26%
Peak instantaneous RPS	77.0	63.5	-18%
p50 at Phase 2	538 ms	596 ms	+11% slower
p50 at Phase 4	767 ms	1,263 ms	+65% slower
Failures	0	0	—

Table 28 - Run 1 vs Run 2 Comparison

Zero failures were recorded across 24,526 total requests. The endpoint degraded gracefully under all load levels without returning HTTP errors. The optimal operating point was 50 concurrent users (~70 avg RPS in Run 1), beyond which throughput plateaued at 52–58 req/s regardless of user count, confirming the server reached its processing ceiling. The sharpest degradation occurred between Phase 2 and Phase 3 in both runs: p99 jumped from ~1,155 ms to ~9,055 ms in Run 1 (8×) and from ~1,504 ms to ~12,367 ms in Run 2 (8×), indicating a concurrency threshold around 50–100 simultaneous connections. Run 2 was measurably slower than Run 1 despite an identical configuration (peak avg RPS dropped 26% and p50 at Phase 4 worsened by 65%), providing direct evidence that returning the full user list without default pagination is a scalability problem that compounds over time.

8.1.2. Smoke Test (POST /users)

Scenario	Expected	Actual	Result
Create valid user	201 + user object	201 {"id": 508999026, "username": "smoke_leonor_01"}	Pass
Create second user	201, different id	201 {"id": 728636236, "username": "smoke_leonor_02"}	Pass
Duplicate username	409 or 400	201 — new user created with a different id every time	Fail
Empty body	422	422 "Field required" at ["body", "username"]	Pass
Missing field	4xx	422 "Field required" at ["body", "username"]	Pass

Table 29 - Functional Tests

Scenario	Actual Response	Result
Empty string username	201 — user created with username: " "	Fail
Whitespace-only username	201 — user created with username: " "	Fail
Integer as username	422 "Input should be a valid string"	Pass

Null username	422 "Input should be a valid string"	Pass
Invalid JSON body	422 "JSON decode error"	Pass
Long username (100 chars)	201 — accepted and stored	No Validation
Extremely long username (1728 chars)	500 — raw DB error leaked	Fail

Table 30 - Functional Tests

Scenario	Actual Response	Result
SQL injection (' ; DROP TABLE users;--)	201 — stored verbatim	ORM prevents execution, but no input validation
XSS payload (<script>alert(1)</script>)	201 — stored verbatim	Not exploitable without a frontend, but persisted in DB
Special characters and spaces	201 — accepted	Undocumented behaviour

Table 31 - Security Tests

ID	Severity	Result
1	High	Duplicate usernames allowed — no uniqueness constraint enforced. smoke_leonor_01 found with 4 different IDs in the database.
2	Medium	Empty string username accepted — {"username": ""} returns 201. Fix: add minLength: 1 to schema.
3	High	Username exceeding 255 chars causes 500 with internal DB schema leaked — STRING(255) has maximum length 255 but got a value with length 1728. Fix: add maxLength: 254 and enforce server-side.
4	Low	No input sanitisation on username — SQL injection and XSS payloads stored verbatim. Not currently exploitable due to ORM and no frontend.
5	Medium	422 response not documented in OpenAPI spec; ErrorResponse.detail defined as string, but actual responses return an array of objects.
6	Medium	GET /users returns a full list without default pagination — measurable performance degradation confirmed across two runs.

Table 32 - Issues Found

The target API demonstrated solid reliability under stress with zero failures across 24,526 requests and graceful degradation at 500 concurrent users. Validation error messages were detailed and well-structured, and SQL injection was not exploitable due to ORM parameterised queries. The main areas requiring improvement are the absence of a uniqueness constraint on username, the lack of

input length validation leading to a 500 with internal schema exposure, and the absence of default pagination on GET /users.

8.2. GET /anime and PUT /users/{user_id} (Joana Carrasqueira)

8.2.1. Stress Test (GET /anime)

The stress test evaluated throughput capacity and response time distribution under concurrent load on the GET /anime endpoint. Locust was configured with 50 virtual users spawned at 5 users/s, a per-user wait time of 0.1–0.5 s between requests, and a 60-second test window. To simulate realistic client behaviour, each virtual user cycled through all 50 combinations of limit $\in \{1, 5, 10, 15, 20\}$ and offset $\in \{0, 20, 40, 60, 80, 100, 120, 140, 160, 180\}$, covering the full pagination surface of the endpoint rather than issuing a single fixed request.

Phase	Users	Spawn Rate
Ramp	20	~5
Ramp	30	~12.5
Ramp	40	~19.7
Ramp	50	~27.3
Steady	50 (steady)	~60–6

Table 33 – Load Ramp Behaviour

Metric	Spawn Rate
Total requests	2882
Failures	0 (0%)
Peak RPS (aggregated)	~67.3
Sustained RPS (steady state)	~65–66
Median response time	440 ms
Average response time	459 ms
Min response time	283 ms
Max response time	1015 ms
percentile 50	440 ms
percentile 75	510 ms
percentile 95	630 ms
percentile 99	740 ms

Table 34 - Stress Test Results Summary

Limit	Offset	Requests	Failures	Median RT	p95 RT	Avg Content size
1	0	98	0	440 ms	600 ms	75 B
1	20	95	0	390 ms	550 ms	120 B
5	0	65	0	460 ms	650 ms	383 B
10	0	50	0	440 ms	620 ms	736 B
15	0	50	0	440 ms	640 ms	1109 B
20	0	50	0	450 ms	680 ms	1526 B
20	60	50	0	450 ms	630 ms	1630 B

Table 35 – Per-Parameter (Limit, Offset) Endpoint Breakdown (representative rows)

Zero failures were recorded across all 2,882 requests. The endpoint remained fully stable throughout the test window. RPS climbed steadily during the spawn ramp and stabilised at approximately 65–67 req/s once all 50 users were active, with no sign of saturation or degradation over time. Response times were consistent across all limit/offset combinations, with the median holding at 440–450 ms and p99 remaining below 750 ms for the duration of steady state.

The single outlier — a maximum of 1,015 ms observed for limit=5/offset=120 — was an isolated spike rather than a trend. Response payload size scaled predictably with limit, from approximately 75 B at limit=1 to 1,630 B at limit=20, confirming correct result slicing with no observable latency impact from larger pages. The evidence suggests the bottleneck at this load level is serialisation and network transfer rather than database query time.

8.2.2. Smoke Test (PUT /users/{user_id})

The smoke test validated the user update endpoint from a functional perspective using Postman against the API. Before every test case, the target user's existence was confirmed via GET /users/{user_id}. After each successful update attempt, the same endpoint was called again to verify that the change had been persisted. Multiple users were created and probed; the cases below represent the most relevant scenarios.

Scenario	user_id	Body	Expected	Actual	Result
Update username user exists (created by tester)	896274687	{"username": "user_updated" }	200 + updated user	500 Internal Server Error	✓
Update username: user exists (user created by another person)	442573965	{"username": "louch"	200 + updated user	200 {"id": 442573965, "username": "louch"}	✓
Update username — user does not exist	999999999999	{"username": "user_updated" }	404 {"detail": "User not found"}	404 {"detail": "User not found"}	✓
Null username (user created by another person)	442573965	{"username": null}	400 {"detail": "Nothing to update"}	400 {"detail": "Nothing to update"}	✓
Empty body (user created by another person)	442573965	{}	400 {"detail": "Nothing to update"}	400 {"detail": "Nothing to update"}	✓
Empty string username (user created by another person)	442573965	{"username": ""}	400 {"detail": "Nothing to update"}	400 {"detail": "Nothing to update"}	✓

Table 36 – Functional Tests

Scenario	user_id	Body	Expected	Actual	Comment
Non-existent user ID	999999999999	{"username": "test"}	404 {"detail": "User not found"}	Passed	Non-existent user ID
Negative user ID	-999999999999	{"username": "test"}	Accepted the value and returned 404 expected error (no validation)	Should reject negative values with 422	Negative user ID
Non-integer user ID (string)	test	{"username": "test"}	422 {"type": "int_parsing", ...}	Passed, should only	Non-integer user ID

				accept int values	(string)
Non-integer user ID (sql path)	1 OR 1=1	{"username": "test"}	422 {"type": "int_parsing", ...}	Passed, should only accept int values	Non-integer user ID (sql path)

Table 37 – Path Parameter Tests

Scenario	Body	Expected	Actual	Comment
Malformed JSON (missing quotes)	{ username: missing_quotes }	422 {"type": "json_invalid", "error": "Expecting property name enclosed in double quotes"}	Passed	Malformed JSON (missing quotes)
Invalid escape sequence	{"username": "x\" UNION SELECT 1,2--"}	422 {"type": "json_invalid", "error": "Invalid \\escape"}	Passed	Invalid escape sequence
Integer as username	{"username": 12345}	422 {"type": "string_type", "msg": "Input should be a valid string"}	Passed, should only accept string values	Integer as username
Boolean as username	{"username": true}	422 {"type": "string_type", "msg": "Input should be a valid string"}	Passed, should only accept string values	Boolean as username
Array as username	{"username": ["admin"]}	422 {"type": "string_type", "msg": "Input should be a valid string"}	Passed, should only accept string values	Array as username
Whitespace-only string	{"username": " "}	200 {"id": 442573965, "username": " "}: accepted and saved	Shouldn't allow username with just whitespaces	Whitespace-only string
XSS payload	{"username": "<script>alert(1)</script>"}	200 {"id": 442573965, "username": "<script>alert(1)</script>"}: accepted and saved literally	Stored safely as string; risk only if rendered in a future frontend	XSS payload
Special characters	{"username": "#\$%&/)(?,-"}	200 {"id": 442573965, "username": "#\$%&/)(?,-"}	Character restrictions on usernames would	Special characters

		"#\$%&/)(?,-": accepted and saved	improve data quality	
Max accepted length (255 chars)	255 × a	200 {"id": 442573965, "username": "aaa..."}: accepted and saved	Accepts up to 255 characters as a username, should be more restrict as well it should be documented in api swagger said limitation	Max accepted length (255 chars)
Over max length (256+ chars)	256 × a	500 Internal Server Error	Fail, 256+ characters are not accepted as a username, but it should return 422 instead of 500	Over max length (256+ chars)

Table 38 – Input Validation Tests

Payload	user_id	Expected	Actual
x' OR '1'='1	442573965	200 {"username": "x' OR '1'='1"}: stored literally	No, parameterized
x' AND 1=2--	442573965	200 {"username": "x' AND 1=2--"}: stored literally	No, parameterized
x' UNION SELECT 1,2--	442573965	422 : JSON parse error, invalid escape	No, blocked at parser
1 OR 1=1 (in path)	1 OR 1=1	422 {"type": "int_parsing"} rejected	No, type validation

Table 39 – SQL Injection Tests

ID	Severity	Result
1	Critical	PUT /users/{user_id} returns 500 when targeting users created during the same session. Users inserted by other groups update correctly, pointing to a server-side bug — likely a missing null check or ORM constraint violation on recently inserted records.
2	High	Usernames of 256 or more characters crash the server with 500 instead of returning 422. The column is likely VARCHAR(255) and the limit is neither enforced server-side nor documented in the spec. Fix: add maxLength: 255 to the OpenAPI schema and validate before the request reaches the database.
3	Low	Negative user IDs (e.g. -9999999999) are accepted by path parameter parsing and fall through to a 404. Explicit rejection with 422 using a minimum: 1 constraint would be more correct.
4	Low	Whitespace-only usernames ({"username": " "}) are accepted and persisted. No minLength is defined in the spec, but a username consisting solely of spaces is likely unintentional data. Server-side

		trimming or a minLength: 1 constraint would prevent this.
5	Info	XSS payloads are stored as literal strings. This is safe at the API layer since responses are JSON, but should be addressed if a frontend is introduced.

Table 40 - Issues Found

The target API demonstrated solid reliability under stress, with zero failures across 2,882 requests and graceful handling of the full pagination surface. SQL injection was not exploitable due to parameterised queries, and FastAPI's built-in validation correctly rejected malformed input before reaching the database. The main areas requiring attention are the 500 triggered by updating recently created users, the missing server-side length cap on the username field, and the absence of default constraints on the user_id path parameter.

8.3. GET /anime/search and DELETE /rating/{id} (Tiago Pina)

8.3.1. Stress Test (GET /anime/search)

The stress test evaluated response time distribution and throughput stability under sustained concurrent load on the /anime/search endpoint. Two independent runs were executed against Group 12's API, each using a flat load profile (not a stepped shape) with zero wait time between requests to maximise pressure on the endpoint.

Phase	Users	Spawn Rate
1	10	2/s
2	50	5/s
3	100	10/s
4	250	25/s
5	500	50/s

Table 41 - Stress Test Parameters

Metric	Value
Total Requests	9,603
Failure Count	1 (99.99% success rate)
Average Throughput	~54.9 req/s
Peak Throughput	76.2 req/s
Average Content Size	~369 bytes
Min Response Time	51 ms
Median (p50)	590 ms
Average Response Time	1,561 ms

95th Percentile (p95)	2,100 ms
99th Percentile (p99)	28,000 ms
Maximum Response Time	101,104 ms (~101 s)

Table 42 - Stress Test Run 1 (200 concurrent users)

Metric	Value
Total Requests	6,695
Failure Count	8 (99.88% success rate)
Average Throughput	~37.3 req/s
Peak Throughput	80.3 req/s
Average Content Size	~366 bytes
Min Response Time	53 ms
Median (p50)	5,900 ms
Average Response Time	11,086 ms
95th Percentile (p95)	39,000 ms
99th Percentile (p99)	47,000 ms
Maximum Response Time	154,829 ms (~154.8 s)

Table 43 - Stress Test Run 2 (500 concurrent users)

Metric	Test 1 (200 Users)	Test 2 (500 Users)	Change
Total Requests	9,603	6,695	-30%
Average Throughput	54.9 req/s	37.3 req/s	-32%
Peak Throughput	76.2 req/s	80.3 req/s	+5%
Median (p50)	590 ms	5,900 ms	10× worse
95th Percentile (p95)	2,100 ms	39,000 ms	18.6× worse
99th Percentile (p99)	28,000 ms	47,000 ms	+68%
Failure Rate	0.01%	0.12%	—

Table 44 - Comparison between Stress Tests

All 9 failures across both runs were 503 Service Unavailable errors emitted by the upstream gateway (upstream connect error or disconnect/reset before headers. reset reason: connection termination), with no application-level exceptions observed in either test.

The endpoint showed acceptable behaviour at 200 users — 95% of requests completed within 2.1 seconds — but degraded severely at 500 users, where the p50

jumped tenfold to 5.9 seconds and p95 reached 39 seconds. Notably, while peak instantaneous throughput was similar between the two runs (~76–80 req/s), average throughput dropped sharply at 500 users (37.3 req/s vs 54.9 req/s) because the growing queue of in-flight requests increased average response times faster than new completions could arrive. The time-series data confirms that the system spent a significant portion of the 500-user run in a queuing regime, with instantaneous RPS oscillating between near-zero and 80 req/s as batches of long-stalled requests resolved together. This points to a concurrency ceiling somewhere between 200 and 500 simultaneous connections — consistent with thread pool exhaustion, database connection pool saturation, or upstream proxy bottlenecks. The gateway 503 errors are a downstream symptom: requests stall in the queue past the proxy connection timeout, after which the gateway severs the connection.

8.3.2. Smoke Test (DELETE /rating/{id})

Testing commenced at 16:15, targeting the DELETE /rating/{id} endpoint. To avoid disrupting data relied upon by other groups, the initial approach was to create a temporary rating using POST /rating, as documented by Group 12.

Scenario	Expected	Actual	Result
Create prerequisite user via POST /user	201 + user object	201 — valid user ID returned	Pass
Retrieve anime list via GET /anime	200 + list	200 — valid IDs returned	Pass
Create rating via POST /rating (valid user + anime ID)	201 + rating object	404 — user not found, despite user confirmed present via GET /user/{id}	Fail
Retrieve existing ratings via GET /rating (after POST failed)	200 + list	Request timeout	Fail
Delete existing rating DELETE /rating/5	204 No Content	404 — rating not found, despite rating visible in subsequent GET	Fail
Retry DELETE /rating/5 (persistent retries until ~17:40)	204 No Content	204 No Content — eventual success	Pass (eventual)
Further POST /rating attempts (until 18:45)	201	Persistent 404 — user validation failure on every attempt	Fail

Table 45 - Smoke tests

ID	Severity	Description
1	High	POST /rating returns 404 claiming the user does not exist despite the user being confirmed present via GET /user/{id}. Points to a database synchronisation issue, stale read, or a bug in the validation query within the rating creation handler.
2	High	DELETE /rating/{id} intermittently returns 404 for a rating that is simultaneously visible via GET /rating. Indicates a state inconsistency or malfunctioning record lookup in the DELETE handler — the endpoint fails to resolve records it can demonstrably find via GET.
3	Medium	GET /rating consistently timed out under concurrent load from multiple testing groups, suggesting the ratings listing query lacks pagination or index coverage and degrades severely under parallel access.

Table 46 - Issues found

8.4. GET /rating and POST /rating (Tiago Pereira)

8.4.1. Stress Test (GET /rating)

The stress test was executed with Locust against the GET /rating endpoint using a stepped load profile (LoadTestShape) progressing from 10 to 500 concurrent users over 5 phases of 60 seconds each, with zero wait time between requests. Unlike the other sections, this endpoint was tested across three separate runs, allowing a more complete comparison of its behaviour under repeated load.

Phase	Users	Spawn Rate	Time Window
1	10	2/s	0–60 s
2	50	5/s	60–120 s
3	100	10/s	120–180 s
4	250	25/s	180–240 s
5	500	50/s	240–300 s

Table 47 - Load Test Phases

Metric	Value
Total requests	604
Failures	40
Median response time	21000 ms
Average response time	24723.06 ms
Max response time	176009.72 ms
Requests/s	2.01

Table 48 - Load Test Phases

Phase	Users	Avg RPS	Max RPS	p50 (ms)	p95 (ms)	p99 (ms)	p99.9 (ms)	Max RT (ms)
1	10	2.2	3.0	3700	4900	5900	5900	11000
2	50	1.8	2.4	3800	22000	25000	25000	50000
3	100	1.9	3.0	12000	29000	43000	50000	54000
4	250	2.4	6.1	19000	41000	87000	88000	104000
5	500	1.8	2.3	19000	44000	98000	104000	173000

Table 49 - Stress Test Run 1

Metric	Value
Total requests	448
Failures	23
Median response time	22000 ms
Average response time	27880.82 ms
Max response time	163927.84 ms
Requests/s	1.57

Table 50 - Stress Test Run 2

Phase	Users	Avg RPS	Max RPS	p50 (ms)	p95 (ms)	p99 (ms)	p99.9 (ms)	Max RT (ms)
1	10	1.5	1.9	5500	6000	6100	6100	6600
2	50	1.5	2.1	5600	18000	25000	26000	50000
3	100	1.4	2.7	12000	38000	65000	69000	69000
4	250	1.5	2.7	17000	60000	85000	89000	108000
5	500	1.7	2.7	21000	70000	141000	147000	164000

Table 51 - Stress Test Run 2

Metric	Value
Total requests	676
Failures	53

Median response time	23000 ms
Average response time	26722.97 ms
Max response time	173992.07 ms
Requests/s	2.27

Table 52 - Stress Test Run 3

Phase	Users	Avg RPS	Max RPS	p50 (ms)	p95 (ms)	p99 (ms)	p99.9 (ms)	Max RT (ms)
1	10	1.5	2.0	5400	6000	6500	6500	6900
2	50	1.9	2.4	5800	18000	25000	26000	47000
3	100	2.4	3.3	15000	34000	55000	63000	63000
4	250	2.9	6.4	21000	43000	76000	88000	111000
5	500	2.6	5.2	23000	54000	133000	140000	167000

Table 53 - Stress Test Run 3

Metric	Run 1	Run 2	Run 3
Total requests	604	448	676
Failures	40	23	53
Median response time	21000 ms	22000 ms	23000 ms
Average response time	24723.06 ms	27880.82 ms	26722.97 ms
Max response time	176009.72 ms	163927.84 ms	173992.07 ms
Requests/s	2.01	1.57	2.27

Table 54 - Run 1 vs Run 2 vs Run 3 Comparison

The three runs showed the same overall pattern. GET /rating remained partially available throughout the test, but response times increased sharply as concurrency grew, and 503 Service Unavailable errors appeared in all three executions. The degradation became much more visible from 100 users onwards, and by the 250- and 500-user phases the high percentiles had already risen to tens of seconds, with the worst cases reaching well above 100 seconds.

Although the exact values varied, the differences between runs were mostly quantitative rather than qualitative. Run 2 had the lowest throughput and the worst average response time, while Run 3 achieved the highest throughput but also

produced the highest failure count. This indicates that the endpoint's behaviour fluctuates somewhat between executions, but the underlying issue remains stable: GET /rating does not scale reliably under progressive concurrent load.

All failures recorded across the three runs were of the same type, 503 Service Unavailable, which reinforces the idea of consistent service saturation rather than isolated or unrelated errors.

8.4.2. Smoke Test (POST /rating)

The smoke test focused on the POST /rating endpoint and was designed to validate whether the API was able to create a new rating under basic functional conditions. The procedure consisted of first confirming that the referenced user_id and anime_id existed through other endpoints, and then submitting a rating creation request with those same values.

Unlike the stress test, this was a functional validation rather than a performance test. The goal was simply to check whether the endpoint behaved correctly with apparently valid input and whether the API accepted new ratings consistently.

Methodology

The test followed these steps:

- Confirm the existence of users through user-related read endpoints in the Postman collection.
- Confirm the existence of anime entries through GET /anime/{id} requests already present in the same collection.
- Submit POST /rating requests with valid-looking payloads using combinations of previously confirmed users and anime.
- The request body used in the collection follows the expected structure for rating creation, including user_id, anime_id, score, and comment fields.

User ID	Anime ID	Expected basis	Observed result
5384650	34264	User and anime were referenced as existing in prior checks	404 Not Found, {"detail": "User not found"}
283642400	38577	User and anime selected from previously validated entities	404 Not Found, {"detail": "User not found"}
769550692	38577	User and anime selected from previously validated entities	404 Not Found, {"detail": "User not found"}
508147556	38577	User and anime selected from previously validated entities	404 Not Found, {"detail": "User not found"}
442573965	38577	User and anime selected from previously validated entities	404 Not Found, {"detail": "User not found"}

Table 55 - Tested Cases

Results

The endpoint did not successfully create any rating during the smoke test, as every observed POST /rating request returned the same 404 response with the message User not found. This behaviour was consistent across multiple payloads and across different user/anime combinations, which makes an isolated bad request less likely as the main explanation.

This is especially relevant because the smoke test was not based on arbitrary IDs. The chosen identifiers had been taken from entities already exposed by other endpoints in the API, so the repeated failure suggests a problem in the validation or integration path used internally by the rating creation flow.⁷

Aspect	Observation
Endpoint tested	POST /rating
Test type	Functional smoke test
Number of observed examples	5 recorded failed requests
Successful creations	0
Repeated error	404 User not found

Table 56 - Observations

Interpretation

From a functional perspective, the smoke test indicates that POST /rating was not operational during the Cloudy Day session. Even though the surrounding API exposed users and anime through read operations, the write operation for creating ratings did not accept those same entities, which points to an inconsistency between read visibility and write-time validation.

A plausible interpretation is that the endpoint depends on an internal validation mechanism, such as cross-service lookup or a separate user resolution step, that was failing even when the external data looked valid. For the purposes of the report, the key conclusion is that the endpoint could not complete its core function under basic smoke-test conditions.

Main takeaway

The smoke test result for POST /rating was negative: no successful rating creation was observed, and all tested requests failed with 404 User not found despite using entities that had already been validated through other API endpoints.

9. Corrections Implemented After Feedback

Two groups (1 and 10) were assigned to test the platform during Cloudy Day (June 3). Both groups executed their tests against the deployed system; however, only Group 10 submitted a formal test report. Group 10's evaluation covered three endpoints through 31 smoke tests and four stress-test scenarios. All 31 smoke tests passed, confirming that the platform's functional contract - including

response codes, data types, field presence, and business rules such as soft-delete filtering - was correctly implemented and observable from an external perspective. No broken links or API contract violations were reported.

The stress tests did surface scalability limitations that had already been identified internally: failures begin to appear at 5 concurrent users, the badges service exhibits anomalously high response times and HTTP 500 errors even under minimal load, and the system experiences widespread HTTP 503 responses under high concurrency due to infrastructure-level saturation.

Since the feedback received was positive with regard to functional correctness, and the scalability limitations identified had been acknowledged internally (single-replica deployments, no HPA configured, badges-service internal bottleneck), no corrective changes were implemented to the platform following the Cloudy Day evaluation. The identified scalability issues are instead documented as future improvements in the conclusions.

10. Cost Analysis

10.1. Monthly Cost

The figures below reflect actual GCP billing from February to May 2026, a period of approximately four months that covers initial cluster provisioning, active development, deployment, and the Cloudy Day evaluation. The monthly cost is derived by dividing total expenditure by four.

Service	Usage Cost (Overall - February to May)	Monthly Cost
Vertex AI	\$0.06	\$0.02
Kubernetes Engine	\$25.57	\$6.39
Cloud Logging	\$0.00	\$0.00
Cloud Storage	\$0.00	\$0.00
Cloud Build	\$0.22	\$0.06
Networking	\$5.48	\$1.37
Cloud Monitoring	\$3.51	\$0.88
Compute Engine	\$55.09	\$13.77
Total	\$89.71	\$22.43

Table 57 - GCP Cost Breakdown (February–May 2026)

The dominant cost categories are Compute Engine (\$55.09) and Kubernetes Engine (\$25.57), which together account for approximately 90% of total expenditure. This is expected given that the platform runs three e2-standard-2 nodes continuously. Vertex AI costs remain negligible (\$0.06 over four months) as the Gemini 2.5 Flash model is only invoked on-demand for review analysis and recommendation explanations, and usage during the project period was low.

10.2. FinOps Measures

Several decisions were made throughout the project to control costs without compromising the required functionality:

- **Cost-effective machine type:** The cluster uses e2-standard-2 nodes (2 vCPU, 8 GB RAM), which are the most price-efficient general-purpose machines available in GCP's E2 family. More capable machine types such as n2-standard-4 were not required given the workloads involved and would have approximately doubled the Compute Engine spend.
- **Single-zone deployment:** The cluster is deployed in a single zone (europe-west1-b) rather than a regional configuration. A regional GKE cluster replicates the control plane and optionally the node pool across three zones, which would have tripled node costs for the same workload. The single-zone trade-off is acceptable for an academic deployment where high availability is not a hard requirement.
- **Right-sized pod resources:** All deployments have explicit CPU and memory requests and limits defined. This prevents any single service from monopolising node resources and allows the Kubernetes scheduler to pack pods efficiently, avoiding the need for additional nodes. The recommendations-service and review-worker, which carry higher computational loads due to Vertex AI SDK overhead, were given proportionally higher allocations while all other services were kept at minimal viable values.
- **Single replica per service:** All microservices run with replicas: 1. While this limits availability, it halves or further reduces the resource footprint compared to running two or more replicas for redundancy, which is appropriate during the development and evaluation phases.
- **Keycloak in development mode:** Keycloak 24 is deployed in start-dev mode using an embedded H2 database. This avoids provisioning a dedicated PostgreSQL instance for Keycloak's internal data, saving the cost and memory overhead of an additional stateful set.
- **Docker Hub for image storage:** Container images are stored in Docker Hub rather than Google Artifact Registry or Google Container Registry. For public images this eliminates registry storage and egress costs entirely, contributing to the \$0.00 Cloud Storage figure.
- **Pay-per-use AI:** Vertex AI (Gemini 2.5 Flash) is invoked only on demand, once per review submission that includes text, and once per recommendation request. There is no always-on model serving infrastructure. The resulting cost of \$0.06 over four months confirms that this architecture is highly cost-efficient for workloads with low or unpredictable AI invocation frequency.
- **Free TLS via Let's Encrypt:** TLS certificates are provisioned automatically by cert-manager using Let's Encrypt, at no cost, rather than purchasing GCP-managed certificates.

11. Conclusions

This project resulted in the successful design, implementation, and cloud deployment of a cloud-native movie platform comprising eight independently deployable microservices, orchestrated on Google Kubernetes Engine. The platform fulfils all 27 functional requirements defined at the outset, exposing a documented REST API across seven services and implementing cross-cutting concerns including JWT-based authentication via Keycloak, gRPC inter-service communication, asynchronous review analysis via GCP Pub/Sub and Vertex AI, Prometheus/Grafana observability, automated TLS provisioning, secret injection via GCP Secret Manager and External Secrets Operator, and a fully scripted deployment pipeline requiring no manual kubectl intervention.

The external testing carried out during Cloudy Day confirmed the functional correctness of the platform. Group 10's evaluation returned 31 out of 31 smoke tests passed across the three evaluated endpoints, validating the API contract, data typing, business rules, and filter behaviour from an independent external perspective. This provides a meaningful signal that the core services behave as intended under normal operating conditions.

The stress tests, however, exposed a clear and significant scalability limitation. The system begins to fail at 5 concurrent users and enters near-total service collapse at 50 users, with a failure rate approaching 98%. Two distinct problems contribute to this. First, the badges service exhibits an internal bottleneck - returning HTTP 500 errors and average response times exceeding 22 seconds even at single-user loads - that is independent of the load on all other services and points to a service-specific issue rather than an infrastructure-wide one. Second, the HTTP 503 responses observed across all services under moderate and high load indicate that the NGINX ingress and pod connection capacity are saturated before the application layer can respond, causing cascading failures. These findings are consistent with the architectural decision to run all services at a single replica with no Horizontal Pod Autoscaler configured, which was identified as a known limitation from the outset.

From a FinOps perspective, the total cost of \$89.71 across four months of active development and deployment translates to approximately \$22.43 per month, driven primarily by Compute Engine and Kubernetes Engine charges. The use of cost-effective e2-standard-2 nodes, a single-zone cluster topology, per-pod resource limits, and on-demand AI invocation kept expenditure well within a manageable range for the scope of the project.

Several improvements are identified for future iterations. Horizontal Pod Autoscaling should be configured for all services, with CPU and memory thresholds calibrated from the load-test data gathered during the evaluation, which would allow the cluster to absorb concurrent traffic spikes without cascading 503 failures. The badges service requires targeted investigation to identify and resolve the internal error causing HTTP 500 responses and the anomalous latency observed even at minimal load. Keycloak should be migrated from start-dev mode with an embedded H2 database to a production configuration backed by a dedicated PostgreSQL instance, as H2 is not suitable for concurrent workloads. Finally, the current single-zone deployment could be upgraded to a regional cluster configuration to eliminate the single point of failure at the infrastructure level should the platform be considered for production use.

Overall, the project successfully demonstrated the design and operation of a cloud-native microservices platform on GCP, integrating modern patterns including asynchronous messaging, AI-powered features, infrastructure-as-code deployment, and CI/CD automation across a cohesive and independently testable service landscape.